

2026 년 KDT 세미 프로젝트 수행계획서

1 조 | 수릿(Surit) — 사진 기반 출장 수리 접수·진행 관리 플랫폼

프로젝트명	수릿 (Surit)
팀 명	1 조
주제	사진 기반 출장 수리 접수 및 진행 관리 플랫폼
기술분야	SW (웹 애플리케이션)
수행기간	2026. 8. 10. ~ 2026. 9. 4. (4 주)
팀 구성	이우진(조장) / 조빌립(형상관리) / 김예원(DB) / 김종혁(이슈·메인) / 박이안(이슈·부) / 오범영(문서) — 총 6 명
성과목표	① Java·SQL·JSP·Spring Boot·MyBatis 를 하나의 완성된 서비스로 통합 ② 브랜치·PR 기반 협업 프로세스 정착 ③ 전 팀원이 서비스 전체 구조와 타 팀원 코드를 설명 가능한 수준 도달
제안배경	출장 수리 접수가 전화 통화에 의존해 증상 전달이 부정확하고 기록이 남지 않으며, 접수 이후 진행상황을 확인할 수단이 없어 고객의 반복 문의와 업체의 응대 부담이 발생한다.
핵심 차별점	① 사진 첨부 기반 증상 접수로 구두 설명의 정보 손실 제거 ② 접수→배정→방문→완료 4 단계 진행상태 실시간 추적 ③ 관리자 기사 배정·일정·비용을 하나의 화면에서 일괄 관리
주요기능	회원가입·세션 로그인 / 수리 접수(사진 업로드) / 접수 내역 검색·페이징 / 진행단계 상세 조회 / 접수 문의 댓글(fetch) / 수리기사 조회 / 관리자 접수 관리·기사 배정·결과 등록
적용 기술	Java 17 / Spring Boot 3.x (내장 Tomcat) / MyBatis / JSP + JSTL / HTML5·CSS3·JavaScript / fetch(AJAX) / Oracle 21c XE / Maven / Git·GitHub
예상 결과물	메인 화면 / 회원가입·로그인 / 수리 접수 폼 / 접수 내역 목록 / 접수 상세(진행단계) / 수리기사 조회 / 관리자 접수 관리 화면
기대효과	접수 정확도 향상 및 정보 누락 방지, 반복 문의 감소, 수리 이력의 체계적 관리, 고장 유형별 통계 기반 운영 개선

II. 프로젝트 수행계획

1. 프로젝트 개요

가. 프로젝트 소개

수릿(Surit)은 고장 난 가전·설비의 출장수리를 온라인으로 접수하고, 처리 진행상황을 단계별로 확인할 수 있는 웹 애플리케이션이다.

고객은 고장 증상과 사진을 함께 등록해 접수하고, 본인 접수 내역의 진행 단계를 언제든지 조회할 수 있다. 관리자는 전체 접수 현황을 확인하고 수리기사 배정, 방문 일정 등록, 처리 결과 및 비용 등록을 수행한다.

Spring Boot 와 MyBatis 기반의 MVC 구조로 구현하며, 화면 전환이 필요한 페이지 이동 및 전체 조회는 Controller 매핑으로, 댓글 등록·기사 배정·상태 변경처럼 화면 전환이 필요 없는 기능은 fetch(AJAX) 비동기 방식으로 구분해 처리한다.

나. 추진배경 및 필요성

- 기존 출장수리 접수는 전화 통화에 의존해 증상이 구두로만 전달되므로 전달 정확도가 낮고 기록이 남지 않는다.
- 접수 이후 진행상황을 확인할 수단이 없어 고객은 반복 문의를 하게 되고, 업체는 그만큼 응대 부담을 떠안는다.
- 증상 사진을 사전에 확보하면 기사가 필요한 부품과 소요 시간을 미리 판단할 수 있어 재방문 가능성을 줄일 수 있다.
- 축적된 접수 데이터를 고장 유형별 통계와 운영 개선의 근거로 활용할 수 있다.

다. 유사 서비스 분석

구분	서비스 성격	수릿과의 차이
숨고	요청서를 등록하면 여러 전문가(고수)가 견적을 보내고 고객이 직접 비교·선택하는 매칭 중개 플랫폼	견적 비교에는 강하지만 매칭 이후의 작업 진행상태를 서비스 안에서 단계별로 추적하는 흐름이 약하다. 수릿은 접수 이후의 진행 추적을 핵심으로 삼는다.
당근	지역 기반 중고거래에서 출발해 동네 업체·생활 정보로 확장된 지역 커뮤니티 플랫폼	동네 업체를 찾는 데는 유용하나 이후 연락과 처리는 개별 채팅·전화로 넘어간다. 수릿은 접수부터 완료까지를 하나의 시스템 안에서 기록으로 남긴다.
수릿	사진 기반 증상 접수 + 관리자 기사 배정 + 4 단계 진행 추적을 하나의 흐름으로 묶은 출장 수리 전용 플랫폼	견적 경쟁이나 지역 커뮤니티가 아니라, 접수 정확도와 진행 투명성에 집중한다.

※ 위 비교는 기획 방향 설정을 위한 정리이며, 발표 전 각 서비스를 팀에서 직접 확인해 최신 기능 기준으로 보완한다.

2. 프로젝트 내용

가. 주요 기능 및 요구사항

ID	구분	기능명	설명	우선순 위
F-01	회원	회원가입 / 로그인	세션 기반 로그인, 일반회원·관리자 권한 구분, 아이디 중복확인(fetch)	상
F-02	회원	마이페이지	회원 정보 수정, 비밀번호 변경, 회원 탈퇴	중
F-03	접수	수리 접수 등록	카테고리·품목·증상 입력, 고장 사진 다중 업로드, 방문 희망일시 및 주소 입력	상
F-04	접수	접수 내역 목록	본인 접수 목록 조회, 상태별 필터, 키워드 검색, 페이징	상
F-05	접수	접수 상세 / 진행단계	접수 상세 정보와 첨부 사진 조회, 접수→배정→ 방문→완료 4 단계 시각화	상
F-06	접수	접수 문의 댓글	접수 건에 대한 문의·답변을 fetch 로 비동기 등록·삭제	중
F-07	기사	수리기사 조회	카테고리·담당 지역별 기사 목록 및 상세 조회	중
F-08	관리자	전체 접수 관리	전체 접수 현황 조회, 상태별 필터, 검색·페이징	상
F-09	관리자	기사 배정 / 방문 일정	접수 건에 담당 기사 배정 및 방문 일정 등록 (fetch 비동기 처리)	상
F-10	관리자	처리 결과 · 비용 등록	수리 내용, 사용 부품, 비용 등록 및 완료 처리	상
F-11	통계	고장 유형별 통계	카테고리별 접수 건수 및 처리 현황 집계 (여유 시 구현)	하

※ 우선순위 "상"은 4 주 내 필수 구현 범위, "중"은 핵심 완료 후 순차 구현, "하"는 일정 여유 시 추가 구현 대상이다.

나. 서비스 흐름도

단계	상태값	고객 행위	관리자 행위
1	접수완료	카테고리·증상·사진·희망일시 입력 후 접수	신규 접수 알림 확인 및 내용 검토
2	기사배정	배정된 기사 정보 및 방문 일정 확인	카테고리·지역 기준으로 담당 기사 배정, 방문 일정 등록
3	방문예정	방문 일정 확인, 문의 댓글 등록	일정 변경 시 상태 갱신
4	처리완료	처리 결과 및 비용 확인	수리 내용·부품·비용 등록 후 완료 처리

3. 적용 기술

기술	적용 내용
Java 17	서버 측 핵심 비즈니스 로직 구현
Spring Boot 3.x	웹 애플리케이션 기본 구조, 의존성 자동 구성, 내장 Tomcat 기반 실행
MyBatis	Oracle DB 와의 SQL 매핑 및 동적 쿼리 처리
JSP + JSTL	서버 사이드 뷰 템플릿 구성 (war 패키징 + tomcat-embed-jasper 의존성 적용)
HTML5 / CSS3 / JavaScript	사용자 화면 및 인터랙션 구현, 공통 default.css 기반 스타일 통일
fetch (AJAX)	댓글 등록·삭제, 아이디 중복확인, 기사 배정 등 화면 전환 없는 비동기 처리
Oracle 21c XE	회원·접수·기사·처리결과 데이터 저장 및 관리
Maven	빌드 및 라이브러리 의존성 관리
Git / GitHub	소스코드 버전 관리, 브랜치·PR 기반 협업

※ Spring Boot 는 Tomcat 을 내장하므로 별도의 외부 Tomcat 설치의 필요하지 않다. 다만 JSP 를 뷰로 사용하려면 패키징을 jar 가 아닌 war 로 지정하고 tomcat-embed-jasper 의존성을 추가해야 하므로, 개발 환경 구축 단계(8/19)에서 이 설정을 최우선으로 검증한다.

4. 필요 기자재 및 개발 도구

품목	활용계획
Spring Tools for Eclipse	Spring Boot 기반 웹 애플리케이션 개발 및 실행·디버깅
Oracle SQL Developer	Oracle DB 및 SQL 관리, 테이블·시퀀스 생성 및 데이터 확인
Figma	UI/UX 와이어프레임 및 화면 설계서 작성
ERDCloud	ERD 작성 및 팀 공유, 구조 변경 이력 관리
Git / GitHub	소스코드 버전 관리, 브랜치 전략 및 PR 기반 코드 검수
Discord	실시간 소통, 데일리 스크럼 및 코드 리뷰 음성 회의
Notion	회의록, 기획 문서, 산출물 아카이브, 작업 보드를 통한 진행 상황 공유

5. 예상 결과물

화면	구성 내용
메인 화면	서비스 소개와 주요 기능 진입 경로를 제공하고, 사용자가 수리 접수 흐름을 쉽게 이해할 수 있도록 구성한다.
회원가입 / 로그인	세션 기반 로그인과 회원가입을 처리하며, 아이디 중복확인 is fetch 로 비동기 검증한다.
수리 접수 폼	카테고리와 증상을 선택하고 고장 사진, 방문 희망일시, 주소를 입력해 출장 수리를 신청한다.
접수 내역 목록	본인이 등록한 접수 내역을 목록으로 확인하고 상태별 필터, 검색, 페이지징으로 원하는 건을 찾는다.
접수 상세 / 진행단계	접수 상세 정보와 첨부 사진을 확인하고, 접수·배정·방문·완료 진행상태를 단계별로 시각화한다.
수리기사 조회	카테고리와 담당 지역 기준으로 수리기사 정보를 조회한다.
관리자 접수 관리	전체 접수 현황 조회, 담당 기사 배정, 방문 일정 등록, 처리 결과 및 비용 등록을 수행한다.

6. 프로젝트 수행방법

가. 추진 일정

구분	추진내용	8/10~11	8/12~18	8/19	8/19~9/1	9/1~9/3	9/3~9/4
계획	주제 선정 및 수행계획서 작성	●					
분석	요구사항 정의, 기능 우선순위 확정	●	●				
설계	ERD 설계, Figma 화면설계, 발표자료 준비		●				
발표	기획발표 및 피드백 반영			●			
환경	개발 환경 구축, Git·DB 초기 세팅			●			
개발	핵심 기능 구현 및 통합				●		
테스트	단위·통합 테스트 및 오류 수정					●	
종료	최종 발표자료 및 시연 준비, 발표						●

나. 개발 우선순위

회원(로그인·세션) → 수리 접수(사진 업로드) → 접수 조회(목록·상세·진행단계) → 관리자 접수 관리 순으로 구현한다. 로그인과 접수가 완성되어야 이후 기능의 데이터가 만들어지므로, 이 두 기능을 개발 첫 주에 반드시 완료한다. 통계 기능은 필수 기능 완료 후 일정 여유가 있을 때만 착수한다.

7. 역할 분담

성명	역할	담당 업무	GitHub
이우진	조장	전체 일정 및 진행 흐름 관리, 팀원별 진행 상황 확인, 멘토·강사 소통	yoell1
조빌립	형상 관리자	Git 저장소 최신 상태 유지, 브랜치·Merge·PR 규칙 관리, 코드 충돌 해결 지원	philip107-jo
김예원	DB 관리자	ERD 최신화, 테이블 설계 관리, DDL 스크립트 및 변경사항 공유	yennie14
김종혁	이슈 관리자(메인)	기술·일정 이슈 관리, 백로그 기록, 이슈 진행 상태 및 해결 여부 확인	Jonyk101
박이안	이슈 관리자(부)	이슈 기록 보조, 미해결 이슈 추적 및 재확인, 이슈 담당자 매칭 지원	seoyeonbag45-spec
오범영	문서 관리자	회의록·주간보고서 작성 및 최신화, 주요 결정사항 기록, 산출물 문서 관리	ohbumyoung

※ 위 역할은 관리 책임 구분이며, 기능 개발은 전원이 분담한다. 담당 기능은 개발 착수 시점(8/19)에 Notion 작업 보드에 등록해 배정한다.

8. 의사소통 방법 및 협업

가. 협업 도구 및 용도

도구	용도
GitHub	소스코드 버전 관리, 브랜치·PR 기반 코드 검수 및 변경 이력 관리
Discord	실시간 소통, 데일리 스크럼, 코드 리뷰 음성 회의, 즉시 공유가 필요한 변경사항 전파
Notion	작업 보드(할 일 / 진행 중 / 완료 / 백로그)로 진행 상황 공유, 회의록·기획 문서·산출물 링크 아카이브

나. 운영 루틴

- 매일 아침 데일리 스크럼을 진행하고, 당일 작업을 1~3 시간 단위로 세분화해 Notion 작업 보드에 등록한다.
- 진행 중(Doing) 상태의 작업은 1 인당 1~2 개로 제한하여 동시 진행 과다를 방지한다.
- 종료 시점에 작업 상태를 갱신하고, 완료하지 못한 작업은 사유를 공유한다.
- 매주 수요일 오전과 금요일 저녁에 전 팀원이 참여하는 코드 리뷰 시간을 갖고, 본인이 작성하지 않은 코드에 대해 질문하고 설명을 듣는다.
- 주간보고서는 매주 월요일 지정 양식을 확인해 최신화하며, 문서 관리자가 취합한다.

다. Ground Rule

- 모든 개발은 본인 브랜치에서 진행하며, main 브랜치에 직접 push 하지 않는다.
- 작업 완료 후 main 의 최신 코드를 내 브랜치로 pull 하여 충돌을 먼저 해결한 뒤 PR 을 생성한다.
- PR 은 최소 1 명 이상의 승인을 받아야 merge 하며, 형상 관리자가 최종 확인한다.
- 커밋 메시지는 "행위 - 내용" 형식(add / update / fix / refactor / merge)으로 50 자 이내로 작성한다.
- 1 시간 이상 해결되지 않는 문제는 혼자 붙들지 않고 팀원에게 공유한다.

- DB 구조 변경이나 공통 코드 수정은 반드시 사전에 공유한 뒤 반영한다.
- 기능 구현 시 본인이 작성한 코드의 동작 방식과 데이터 흐름을 설명할 수 있도록 한다.

라. 품질 관리

- 기능 개발 후 정상 동작 여부를 확인하고, 오류 원인과 수정 내용을 팀에 공유한다.
- 공통 스타일은 default.css 를 기준으로 통일하고, 개별 화면 스타일만 추가한다.
- 최종 단계에서 고객 접수부터 관리자 처리 결과 등록까지 전 과정을 통합 테스트한다.

III. 기대효과 및 활용분야

1. 기대효과

가. 고객 측면

- 전화 설명 과정의 정보 누락을 줄이고 사진과 입력 정보를 함께 전달하여 접수 정확도를 높일 수 있다.
- 접수 진행단계를 직접 조회할 수 있어 반복 문의와 대기 불편을 줄일 수 있다.
- 방문 희망일시와 주소를 사전에 등록하여 접수 편의성을 높일 수 있다.

나. 업체·관리자 측면

- 접수 내용을 시스템에 기록하여 수리 이력을 체계적으로 관리할 수 있다.
- 전체 접수 현황, 담당 기사, 방문 일정, 처리 결과 및 비용을 한곳에서 관리하여 업무 누락을 줄일 수 있다.
- 사전에 확보한 증상 사진으로 필요한 부품과 소요 시간을 예측해 재방문 가능성을 낮출 수 있다.

다. 팀 측면

- 브랜치·PR 기반 협업 프로세스를 실제 프로젝트에서 경험하여 실무형 협업 역량을 확보한다.
- 주 2 회 코드 리뷰를 통해 전 팀원이 서비스 전체 구조를 이해하고 설명할 수 있는 상태에 도달한다.

2. 활용분야

활용분야	활용 방법
가전 수리	가전제품 고장 접수 및 출장 방문 일정 관리
보일러·에어컨·설비	증상·사진 기반 접수, 기사 배정 및 처리 결과 관리
인터넷 설치 등 출장 서비스	방문 주소와 희망일시 기반 출장 접수·관리
아파트·임대주택	시설 하자 접수와 처리 상태 조회 시스템으로 확장
사내 IT 헬프데스크	사내 장비 고장 접수 및 담당자 배정·처리 이력 관리